



Extreme Learning Machine with Kernels for Solving Elliptic Partial Differential Equations

Shaohong Li¹ · Guoguo Liu² · Shiguo Xiao³

Received: 30 April 2021 / Accepted: 8 May 2022 / Published online: 10 June 2022
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

Finding solutions for partial differential equations based on machine learning methods is an ongoing challenge in applied mathematics. Although machine learning methods have been successfully applied to solve partial differential equations, their practical applications are limited by a large number of variables. In this study, after a strict theoretical derivation, a new method is proposed for solving standard elliptic partial differential equations based on an extreme learning machine with kernels. The parameters of the proposed method (i.e., regularization coefficients and kernel parameters) were obtained by a grid search approach. Three numerical cases combined with some comprehensive indices (mean absolute error, mean squared error and standard deviation) were used to test the performance of the proposed method. The results show that the performance of the proposed method is superior to that of existing methods, including the wavelet neural network optimized with the improved butterfly optimization algorithm. In addition, the proposed method has fewer unknown parameters than previous methods, which makes its calculations more convenient. In this study, the effect of the number of training points on the calculation results is also discussed, and the advantage of the proposed method is that only a few training points are needed to achieve high computational accuracy. In addition, as a case study, the proposed method is successfully applied to simulate the water flow in unsaturated soils. The proposed method provides new insight for solving elliptic partial differential equations.

Keywords Elliptic partial differential equations · Numerical solution · Machine learning · Extreme learning machine with kernels

Introduction

As an important bridge between analytic mathematics and applied science, partial differential equations have been widely used in applied mathematics [1–4]. Machine learning

has been an intensively studied research area in recent years; the methods studied include artificial neural networks, support vector machines, and extreme learning machines, which have been developed and used in practical applications [5–7]. Due to their good nonlinear approximation abilities and versatility, solving partial differential equations (PDEs) based on machine learning methods has attracted increasing attention [8].

Lagaris et al. [9] pioneered the use of artificial neural network methods to obtain solutions to ordinary differential equations (ODEs) and PDEs. Trial solutions to ODEs and PDEs were constructed using neural networks, and then, solutions were transformed into optimization problems. The parameters of the neural network were obtained through a gradient-based learning algorithm. Lagaris et al. [10] further developed a neural network-based method for solving PDEs with irregular boundaries. Alli et al. [11] used an improved back-propagation neural network to solve ODEs and PDEs

✉ Guoguo Liu
ldamienrice@my.swjtu.edu.cn

✉ Shiguo Xiao
xiaoshiguo@swjtu.cn

¹ Department of Geological Engineering, Southwest Jiaotong University, Chengdu 610031, China

² School of Civil Engineering, Southwest Jiaotong University, Chengdu 610031, China

³ Ministry of Education, Key Laboratory of High-Speed Railway Engineering, Southwest Jiaotong University, Chengdu 610031, China

in vibration control problems, which was validated by analytical and Runge–Kutta methods. Filici [12] focused on solving ODEs with neural networks using an improved Levenberg–Marquardt algorithm to learn the parameters of the neural network. Several examples demonstrate that the proposed method can obtain high-precision solutions. Effati and Pakdaman [13] pointed out that neural network-based methods can obtain better results than existing methods when solving nonlinear fuzzy ODEs. Mall and Chakraverty [29] proposed a method based on a single layer Chebyshev neural network and successfully obtained solutions to elliptic PDEs. This method uses fewer parameters than traditional neural networks. To prevent the gradient-based learning algorithm from falling into local optimization, Tan et al. [8] developed an improved butterfly optimization algorithm (one of the heuristic optimization algorithms) to learn the parameters of the wavelet neural network. However, due to the numerous parameters of neural networks, learning methods based on the heuristic optimization algorithm are computationally expensive. In addition, some new approaches (e.g., [14–16] have been developed to solve optimization problems that provide an important reference for solving PDEs by machine learning methods.

An extreme learning machine (ELM) is a single hidden layer neural network that was first proposed by Huang et al. [17] and can be regarded as a special case of neural networks. Compared with other learning algorithms of neural networks, the learning algorithm of an ELM is simpler and more efficient [18]. Therefore, ELMs have been widely used in many practical applications, including landslide displacement prediction [19–21] and signal processing [22]. However, the performance of ELMs depends on the weight of the input layer and the number of neurons in the hidden layer, which often makes its performance unstable [23]). To increase the generalization performance and stability of ELM, an extreme learning machine with kernels (KELM) was further developed [24] and has been widely used in various engineering problems. For example, Zhou et al. [25] proposed a landslide displacement prediction method based on a KELM. A case study showed that the accuracy of the KELM was higher than that of an ELM, support vector regression, and backward propagation neural networks. Alizamir et al. [26] compared the performances of KELM, boosted regression tree, radial basis function neural network, multilayer perceptron neural network, and multivariate adaptive regression spline models in estimating daily dew point temperatures. Several examples show that the accuracy of the KELM is higher than that of the remaining models. Li et al. [27] developed a hybrid method based on a KELM to predict the water level of rivers, which was validated by prediction cases of the Xiangjiang River and Yuanjiang River. Chen et al. [28] developed a combined method for predicting the displacement of the Jinping–I arch dam in China, and a KELM was used as a benchmark model for comparison.

Existing studies have confirmed the outstanding performance of KELMs. However, there are few studies on solving partial differential equations with KELMs. The objective of this paper is to propose an improved method to solve elliptic PDEs based on a KELM. The rest of this paper is organized as follows:

The “**Propaedeutics: a Brief Overview of KELMs**” section briefly presents the basic theory underpinning KELMs. The “**Solving an Elliptic PDE using KELMs**” section introduces, in detail, the proposed method for solving elliptic PDEs based on a KELM, and the concrete calculation steps are also given. In “**Numerical Examples and Discussion**” section, numerical analysis results are used to compare the accuracy of the proposed method with the accuracies of other existing methods. The “**Application**” section describes the application of the proposed method to evaluate water flow in unsaturated soils. Conclusions are presented to end the paper.

Propaedeutics: a Brief Overview of KELMs

Considering a multi-input, single-output problem, the input sample is denoted $X = (X_j)_{j=1,2,\dots,N}$ (N represents the number of samples), while the output sample is denoted $T = (t_j)_{j=1,2,\dots,N}$. A single hidden layer neural network (Fig. 1) can be mathematically modeled as [17]

$$o_j = \sum_{i=1}^L \beta_i g(W_i \cdot X_j + \kappa_i) \approx t_j, j = 1, \dots, N \quad (1)$$

$$g(x) = \frac{1}{1 + \exp(-x)} \quad (2)$$

where o_j is the output variable of the ELM, β_i is the weight of the output layers, $g(x)$ is an activation function, W_i is the weight between the input layers and the i th hidden neuron, κ_i is the threshold of the i th hidden neuron, ‘ $\cdot$ ’ is the inner

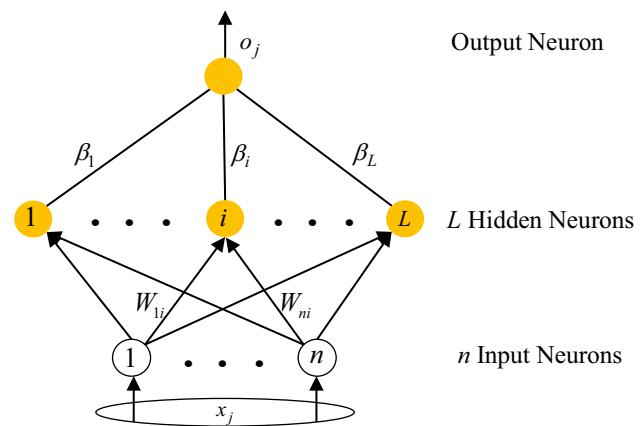


Fig. 1 Network structure of the extreme learning machine

product symbol, and L represents the number of neurons in the hidden layers.

The aim of ELM learning is to minimize the output error, i.e.,

$$\sum_{j=1}^N \|o_j - t_j\| = 0 \quad (3)$$

Substituting Eq. (1) into Eq. (3) leads to

$$\sum_{i=1}^L \beta_i g(W_i \cdot X_j + \kappa_i) = t_j, j = 1, \dots, N \quad (4)$$

Equation (4) can be rewritten as

$$H\beta = T \quad (5)$$

where

$$H = \begin{bmatrix} g(W_1 \cdot X_1 + \kappa_1) & g(W_2 \cdot X_1 + \kappa_2) & \dots & g(W_L \cdot X_1 + \kappa_L) \\ g(W_1 \cdot X_2 + \kappa_1) & g(W_2 \cdot X_2 + \kappa_2) & \dots & g(W_L \cdot X_2 + \kappa_L) \\ \vdots & \vdots & \ddots & \vdots \\ g(W_1 \cdot X_N + \kappa_1) & g(W_2 \cdot X_N + \kappa_2) & \dots & g(W_L \cdot X_N + \kappa_L) \end{bmatrix}_{N \times L}$$

and $\beta = \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_L \end{bmatrix}_{L \times 1}, T = \begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_N \end{bmatrix}_{N \times 1}.$

In the ELM, once the input layer weight W and hidden layer threshold κ are randomly given, the estimated output weight β can be determined:

$$\beta = H^\dagger T \quad (6)$$

where $H^\dagger$ is the Moore–Penrose generalized inverse of the matrix H . Equation (6) shows that training an ELM is simply equivalent to solving a linear system.

To improve the stability and generalization ability of the ELM, according to Tikhonov regularization and ridge regression, a regularization coefficient η is introduced into Eq. (6) and leads to.

$$\beta = H^T (HH^T + \eta I)^{-1} T \quad (7)$$

where I is the unit matrix. Considering that the activation function is unknown, the kernel function is introduced in Eq. (7) to obtain

$$\beta = H^T (\Omega + \eta I)^{-1} T \quad (8)$$

$$\Omega = \{K(X_i, X_j)\}_{i,j=1,2,\dots,N} \quad (9)$$

$$K(X_i, X_j) = \exp\left(-\frac{\|X_i - X_j\|^2}{\gamma^2}\right) \quad (10)$$

where $K(X_i, X_j)$ is a Gaussian kernel function and γ is a kernel parameter.

Correspondingly, the simulated value can be written as

$$o_j = H\beta = HH^T (HH^T + \eta I)^{-1} T$$

$$= \begin{bmatrix} K(X_j, X_1) \\ K(X_j, X_2) \\ \vdots \\ K(X_j, X_N) \end{bmatrix}^T (\Omega + \eta I)^{-1} T \quad (11)$$

Equation (11) is the expression of the KELM. Comparing Eq. (1) with Eq. (11) and the ELM with the KELM shows that the KELM does not need to specify the weight of the input layer or the number of neurons in the hidden layer.

Solving an Elliptic PDE Using KELMs

A standard elliptic PDE is considered,

$$\frac{\partial^2 z}{\partial y^2} + \frac{\partial^2 z}{\partial x^2} = f(x, y, z), a \leq x \leq b, c \leq y \leq d, \quad (12)$$

with Dirichlet boundary conditions

$$z(a, y) = h_0(y), z(b, y) = h_1(y), z(x, c) = q_0(x), z(x, d) = q_1(x).$$

Using KELMs to solve Eq. (12) consists of two steps. First, the trial solution is represented as a combination of boundary conditions and a KELM output. Second, the parameters of the trial solution are calculated.

The trial solution of Eq. (12) can be expressed as follows [9]:

$$z(x, y) = A(x, y) + B(x, y)u(x, y), \quad (13)$$

where

$$A(x, y) = \frac{b-x}{b-a}h_0(y) + \frac{x-a}{b-a}h_1(y) + \frac{d-y}{d-c} \left\{ q_0(x) - \left[\frac{(b-x)}{b-a}q_0(a) + \frac{(x-a)}{b-a}q_0(b) \right] \right\} \\ + \frac{(y-c)}{d-c} \left\{ q_1(x) - \left[\frac{(b-x)}{b-a}q_1(a) + \frac{(x-a)}{b-a}q_1(b) \right] \right\},$$

$B(x, y) = (x-a)(b-x)(y-c)(d-y)$, and u is the output of the KELM.

Based on “Propaedeutics: a Brief Overview of KELMs” section, the error function for solving Eq. (12) can be written as

$$\sum_{j=1}^N \left\| \frac{\partial^2 z_j}{\partial y^2} + \frac{\partial^2 z_j}{\partial x^2} - f(x_j, y_j, z_j) \right\| = 0 \quad (14)$$

The first/second-order derivative in Eq. (14) can be obtained according to the following equations:

$$\frac{\partial z}{\partial x} = \frac{\partial A}{\partial x} + \frac{\partial B}{\partial x}u + \frac{\partial u}{\partial x}B \quad (15)$$

$$\frac{\partial z}{\partial y} = \frac{\partial A}{\partial y} + \frac{\partial B}{\partial y}u + \frac{\partial u}{\partial y}B \quad (16)$$

$$\frac{\partial^2 z}{\partial x^2} = \frac{\partial^2 A}{\partial x^2} + \frac{\partial^2 B}{\partial x^2}u + 2\frac{\partial B}{\partial x}\frac{\partial u}{\partial x} + \frac{\partial^2 u}{\partial x^2}B \quad (17)$$

$$\frac{\partial^2 z}{\partial y^2} = \frac{\partial^2 A}{\partial y^2} + \frac{\partial^2 B}{\partial y^2}u + 2\frac{\partial B}{\partial y}\frac{\partial u}{\partial y} + \frac{\partial^2 u}{\partial y^2}B \quad (18)$$

Substituting Eqs. (15) and (16) into Eq. (14), the PDEs can be transformed into a linear system problem according to “Propaedeutics: a Brief Overview of KELMs” section:

$$\sum_{j=1}^N \left\| \begin{aligned} &\frac{\partial^2 A(x_j, y_j)}{\partial x^2} + \frac{\partial^2 A(x_j, y_j)}{\partial y^2} + \left(\frac{\partial^2 B(x_j, y_j)}{\partial x^2} + \frac{\partial^2 B(x_j, y_j)}{\partial y^2} \right) u(x_j, y_j) \\ &+ 2\frac{\partial B(x_j, y_j)}{\partial x} \frac{\partial u(x_j, y_j)}{\partial x} + 2\frac{\partial B(x_j, y_j)}{\partial y} \frac{\partial u(x_j, y_j)}{\partial y} \\ &+ \frac{\partial^2 u(x_j, y_j)}{\partial x^2} B(x_j, y_j) + \frac{\partial^2 u(x_j, y_j)}{\partial y^2} B(x_j, y_j) - f(x_j, y_j, z_j) \end{aligned} \right\| = 0 \quad (19)$$

Borrowing the idea of an extreme learning machine with kernels and denoting $\mathbf{X} = (x_j, y_j)_{j=1,2,\dots,N}$, u can be expressed as

$$u_j = K(\mathbf{X}_j, \mathbf{X})(\eta \mathbf{I} + \Omega)^{-1} T_j, j = 1, 2, \dots, N, \quad (20)$$

where

$$T_j = - \left[\frac{\partial^2 A(X_j)}{\partial x^2} + \frac{\partial^2 A(X_j)}{\partial y^2} - f(X_j, z_j) \right], \quad (21)$$

$$\Omega = \{\Omega_{ij}\}_{i,j=1,2,\dots,N}, \text{ and} \quad (22)$$

$$\Omega_{ij} = K(X_i, X_j) + 2\frac{\partial B(X_j)}{\partial x} \frac{\partial K(X_i, X_j)}{\partial x} + 2\frac{\partial B(X_j)}{\partial y} \frac{\partial K(X_i, X_j)}{\partial y} + \frac{\partial^2 K(X_j)}{\partial x^2} B(x_j, y_j) + \frac{\partial^2 K(X_j)}{\partial y^2} B(x_j, y_j) \quad (23)$$

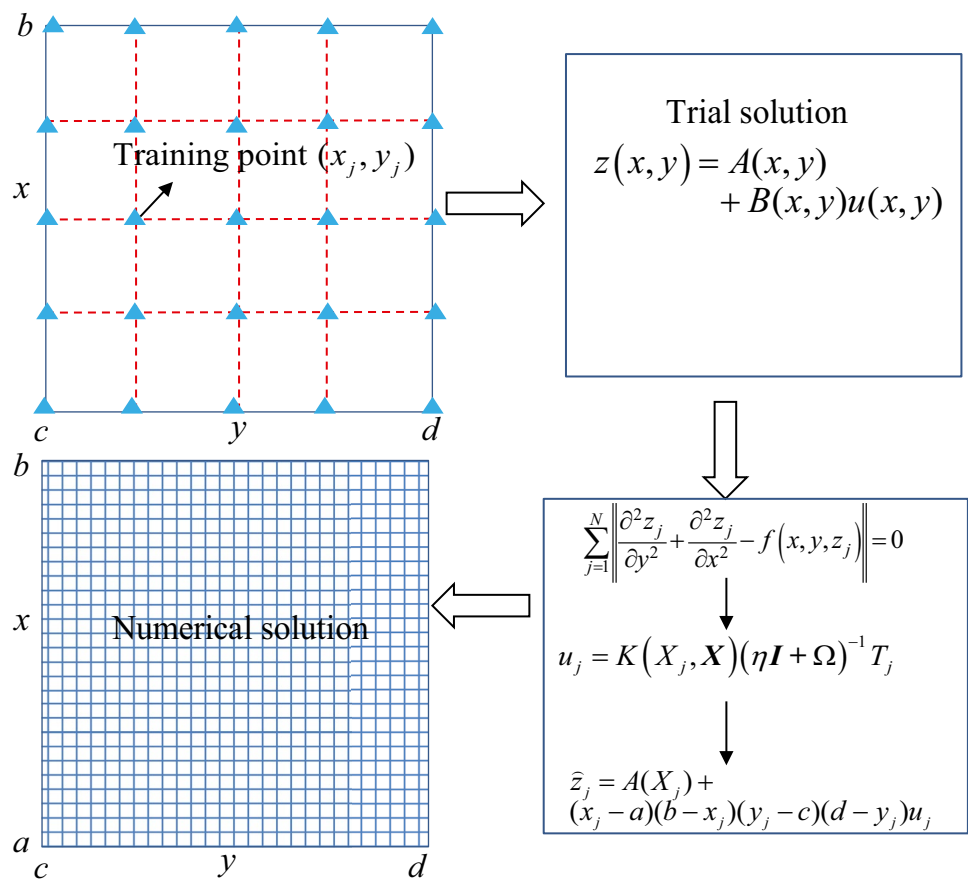
Ω_{ij} and T_j can be computed according to Appendix A1.

According to Eq. (13), the numerical solution to Eq. (12) can be expressed as

$$\hat{z}_j = A(X_j) + (x_j - a)(b - x_j)(y_j - c)(d - y_j)u_j \quad (24)$$

The time complexity of the proposed method mainly depends on the term $(\eta \mathbf{I} + \Omega)^{-1}$ in Eq. (20), Eq. (20) can

Fig. 2 Flow chart of the proposed method



be rewritten as $u_j = K(X_j, X)W = K(X_j, X)(\eta I + \Omega) \setminus T_j$, and solving variable W is equivalent to solving a linear system of equations, the time complexity of which is $O(N^3)$.

Figure 2 shows the flow chart of the proposed model. According to Fig. 2, the proposed method can obtain a closed solution. Detailed steps for solving Eq. (12) are given as follows:

Step 1: Select N training points in the rectangular area $[a, b] \times [c, d]$.

Step 2: Construct trial solutions based on boundary conditions.

Step 3: Solve Eq. (20).

Step 4: Calculate the numerical solution to Eq. (12) based on Eq. (24).

Note that the KELM contains two parameters (i.e., the regularization coefficient η and kernel parameter γ) that need to be specified. In this paper, a grid search method was used to determine these two parameters (see “Numerical Examples and Discussion” section for details).

Numerical Examples and Discussion

The experiments were conducted in MATLAB 2018a (Windows 10 and an Intel i5-8265U processor with 8 GB RAM). In the following three cases, the domains considered were $[0, 1] \times [0, 1]$.

Example 1 (Mall and Chakraverty [29]; Tan et al. [8])

We considered the following PDE:

$$\frac{\partial^2 z}{\partial y^2} + \frac{\partial^2 z}{\partial x^2} = e^{-x}(x - 2 + y^3 + 6y), \quad 0 \leq x \leq 1, 0 \leq y \leq 1 \quad (25)$$

with the Dirichlet boundary condition

$$z(0, y) = y^3, z(1, y) = (1 + y^3)e^{-1}, z(x, 0) = xe^{-x}, z(x, 1) = (x + 1)e^{-x}.$$

According to Eq. (13), the trial solution to Eq. (25) can be expressed as

$$z = A(x, y) + x(1 - x)y(1 - y)u(x, y)$$

$$\text{where } A(x, y) = (1 - x)y^3 + x(1 + y^3)e^{-1} + x(1 - y)(e^{-x} - e^{-1}) + y[(1 + x)e^{-x} - (1 - x + 2xe^{-1})].$$

The exact solution to Eq. (25) is $z = e^{-x}(x + y^3)$ (Fig. 3).

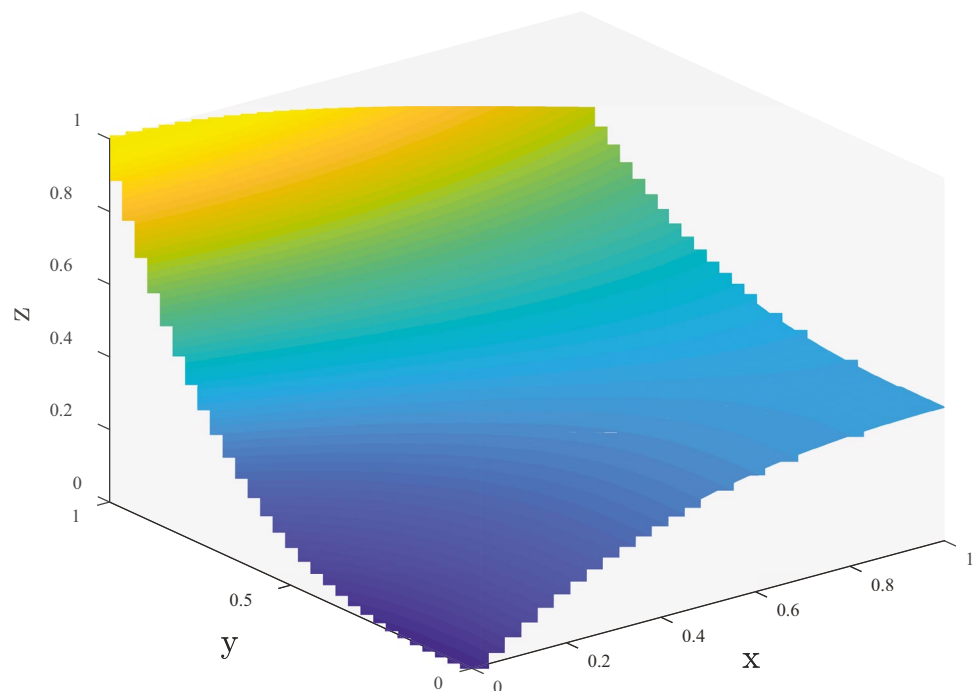
To quantitatively compare the accuracies of the proposed method and the existing methods, the absolute error (AE), mean absolute error (MAE), mean squared error (MSE), and standard deviation (SD) were selected as the quantitative indices, which are defined as

$$AE = |\hat{z}_j - z_j| \quad (26)$$

$$MAE = \frac{1}{N} \sum_{j=1}^N |\hat{z}_j - z_j| \quad (27)$$

$$MSE = \frac{1}{N} \sum_{j=1}^N (\hat{z}_j - z_j)^2 \quad (28)$$

Fig. 3 Exact solution to Eq. (25)



$$SD = \sqrt{\frac{1}{N} \sum_{j=1}^N (AE - MAE)^2} \quad (29)$$

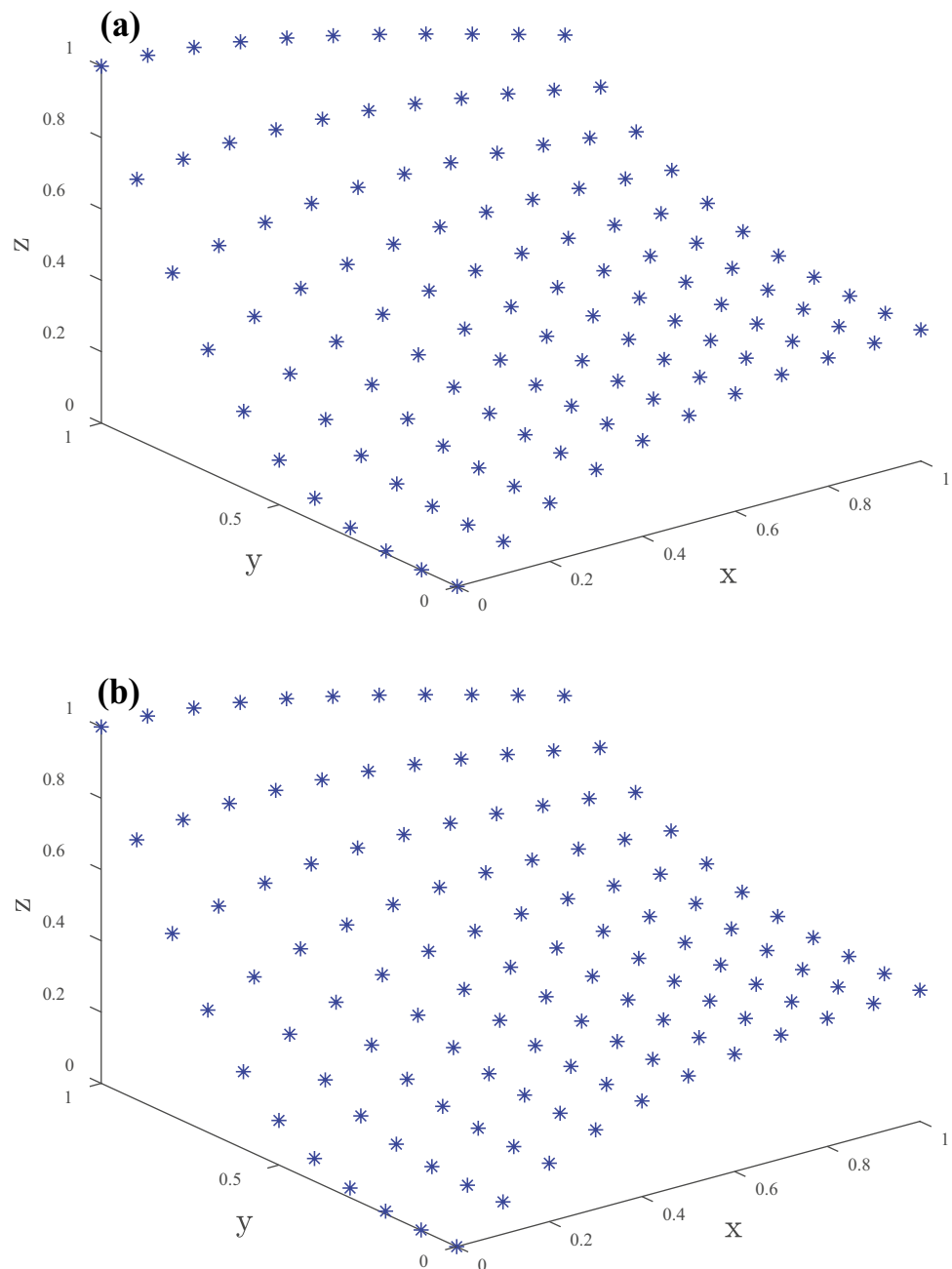
where z_j represents the j th exact solution and z_{mean} represents the average value of the numerical solutions. Among the above four indices, the smaller the AE, MAE, and MSE are, the more accurate the solution of the method; the smaller the SD is, the better the robustness of the method.

In this example, the methods used for comparison with the proposed method include the Chebyshev neural network

(ChNN) [29], wavelet neural network optimized with the particle swarm optimization algorithm (WNNPSO) [8], wavelet neural network optimized with the butterfly optimization algorithm (WNNBOA) [8], and wavelet neural network optimized with the improved butterfly optimization algorithm (WNNIBOA) [8] (these methods are briefly introduced in [Appendix A2](#)).

A 10×10 grid was used to discretize the domain, and then, 121 points were selected as training points. In this work, the parameters of the KELM (regularization coefficient η and kernel parameter γ) were selected from the set

Fig. 4 Solutions to training points in Example 1: **a** exact solution and **b** numerical solution of the proposed model



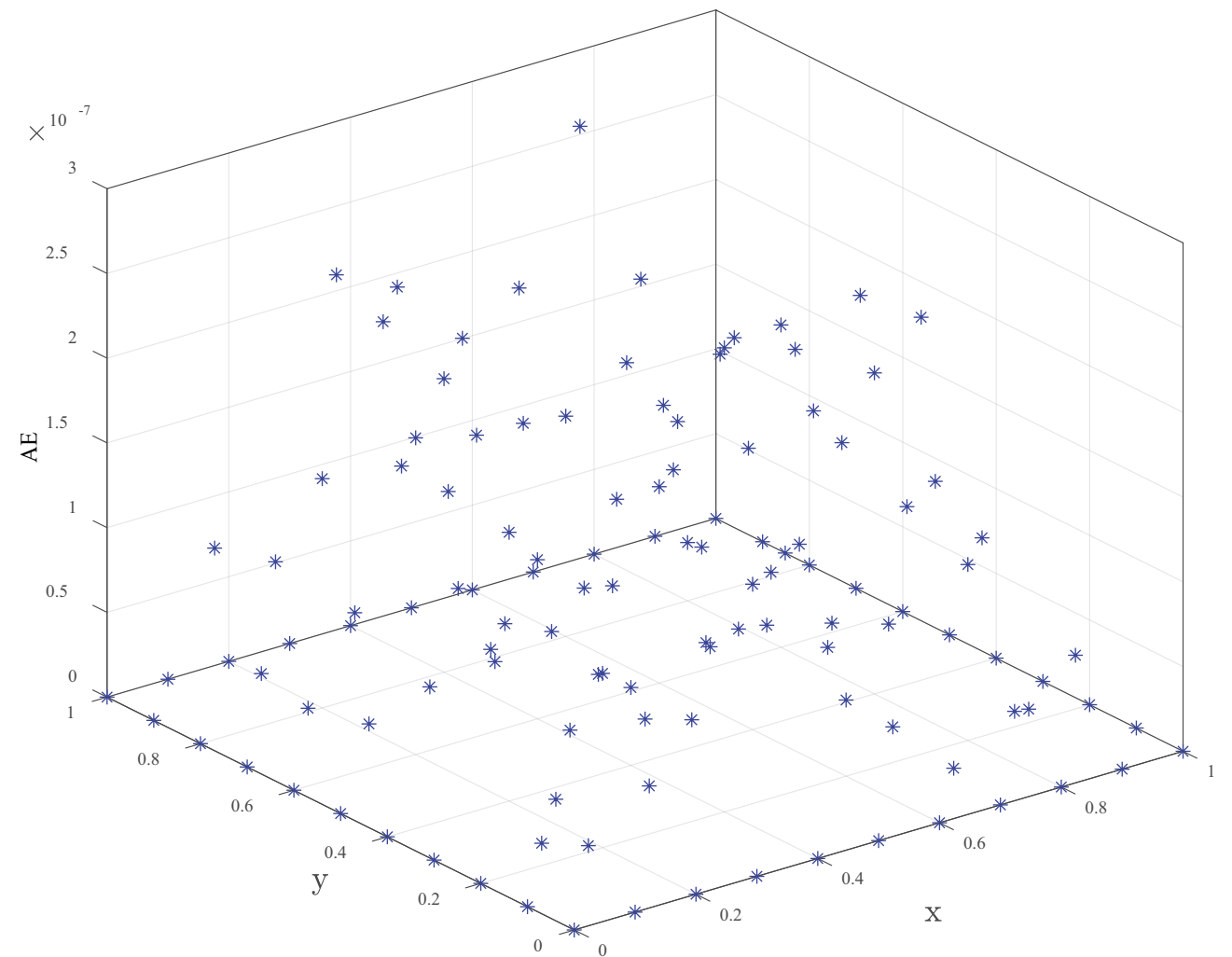


Fig. 5 Absolute error values of training points in Example 1

of values $\{2^i | i = -9, -8, \dots, 10\}$ by tuning a set comprising a random 10% of the sample set (i.e., by using the grid search method) [30]. Figure 4 reports the exact solutions and KELM solutions at these points. The demo code can be found at [https://github.com/lishao12/KELM-for-solving-](https://github.com/lishao12/KELM-for-solving-elliptic-PDE)

[elliptic-PDE](https://github.com/lishao12/KELM-for-solving-elliptic-PDE). Figure 5 reports the AE values of the KELM. Figure 5 shows that the AE values of the KELM are all less than 10^{-6} . Since the trial solution automatically meets the boundary conditions, the AE at the boundary conditions is 0. The maximum value of the AE appears in the

Table 1 Absolute errors of the five methods

<i>x</i>	<i>y</i>	ChNN [29]	WNNPSO [8]	WNNBOA [8]	WNNIBOA [8]	KELM
0	0.2318	0	0	0	0	0
0.2174	0.749	1.30E-02	6.51E-06	1.05E-03	9.87E-06	3.42E-07
0.587	0.3285	3.20E-02	6.12E-06	1.70E-03	8.46E-06	1.14E-06
0.3971	0.6481	2.60E-03	1.36E-05	8.05E-04	6.99E-06	7.00E-07
0.7193	0.2871	2.60E-02	6.95E-07	6.10E-04	1.16E-05	8.63E-07
0.8752	0.538	1.60E-03	2.95E-05	3.31E-05	3.85E-07	4.53E-07
0.9471	0.4691	7.60E-03	4.87E-05	1.85E-04	3.31E-06	8.66E-07
0.4521	0.8241	1.50E-02	4.57E-05	1.28E-03	1.25E-05	1.81E-07
0.298	0.9153	5.50E-03	1.77E-05	1.10E-03	2.51E-05	9.08E-07
0.632	0.1834	2.30E-02	2.40E-05	5.05E-04	1.94E-05	4.97E-06

central area of the solution domain. According to Eqs. (24) and (25), the MAE, MSE, and SD values at these training points are approximately $1.02 \text{ E-}07$, $2.85\text{E-}14$, and $1.17\text{E-}07$, respectively.

The untrained points were adopted to further compare the performances of the KELM, ChNN, WNNPSO, WNNBOA, and WNNIBOA (the calculation results of ChNN are from Mall and Chakraverty [29]; the calculation results of the WNNPSO, WNNBOA, and WNNIBOA are from Tan et al. [8]. These sampling points are the same as those reported in the literature to fairly evaluate the performance of each scheme. Because the output layer weights of the ChNN, WNNPSO, WNNBOA, and WNNIBOA are obtained through iteration, the parameters of the KELM are less than those of these four methods. Table 1 shows the AEs of the five methods at each test point. As shown in Table 1, the AEs of the KELM range from 10^{-7} to 10^{-6} , the AEs of the WNNPSO vary between 10^{-7} and 10^{-5} , the AEs of the WNNBOA vary between 10^{-5} and 10^{-3} , the AEs of the WNNIBOA vary between 10^{-7} and 10^{-5} , and the AEs of the ChNN are between 10^{-3} and 10^{-2} . According to Table 1, the MAEs of the ChNN, WNNPSO, WNNBOA, WNNIBOA, and KELM are $1.26\text{E-}02$, $1.93\text{E-}05$, $7.27\text{E-}04$, $9.76\text{E-}06$, and $1.04\text{E-}06$, respectively. The MSEs of the ChNN, WNNPSO, WNNBOA, WNNIBOA, and KELM are $2.72\text{E-}04$, $6.48\text{E-}10$, $8.15\text{E-}07$, $1.53\text{E-}10$, and $2.92\text{E-}12$, respectively. The SDs of the ChNN, WNNPSO, WNNBOA, WNNIBOA, and KELM are $1.06\text{E-}02$, $1.67\text{E-}05$, $5.36\text{E-}04$, $7.57\text{E-}06$, and $1.35\text{E-}06$, respectively. The performances of the five methods from high to low are in the order the KELM, WNNIBOA, WNNPSO, WNNBOA, and ChNN.

Example 2 (Tan et al. [8])

We considered the following PDE:

$$\frac{\partial^2 z}{\partial y^2} + \frac{\partial^2 z}{\partial x^2} = -z(x^2 + y^2) \quad 0 \leq x \leq 1, 0 \leq y \leq 1 \quad (30)$$

with Dirichlet boundary conditions

$$z(0, y) = 0, z(1, y) = \sin y, z(x, 0) = 0, z(x, 1) = \sin x.$$

According to Eq. (13), the trial solution to Eq. (30) is expressed as

$$z = A(x, y) + x(1-x)y(1-y)u(x, y)$$

where $A(x, y) = y \sin x + x \sin y - xy \sin 1$.

The exact solution is $\hat{z} = \sin(xy)$ (Fig. 6).

In this example, the WNNPSO, WNNBOA, and WNNIBOA were compared with the KELM.

The sampling points are the same as those in Example 1. Figure 7 reports the exact solutions and KELM solutions at these points. Figure 8 reports the AE values of the KELM. Figure 8 shows that the AE values of the KELM are almost less than 10^{-6} . Similar to Example 1, the AE value at the boundary is 0 because the trial solution automatically satisfies the boundary conditions. The MAE, MSE, and SD values at the training point are approximately $5.23 \text{ E-}07$, $1.14\text{E-}13$, and $3.78\text{E-}07$, respectively.

Table 2 shows the performance comparison of the KELM, WNNPSO, WNNBOA, and WNNIBOA on untrained points (the calculation results of the WNNPSO, WNNBOA, and WNNIBOA are from Tan et al. [8]. Similar to

Fig. 6 Exact solution to Eq. (30)

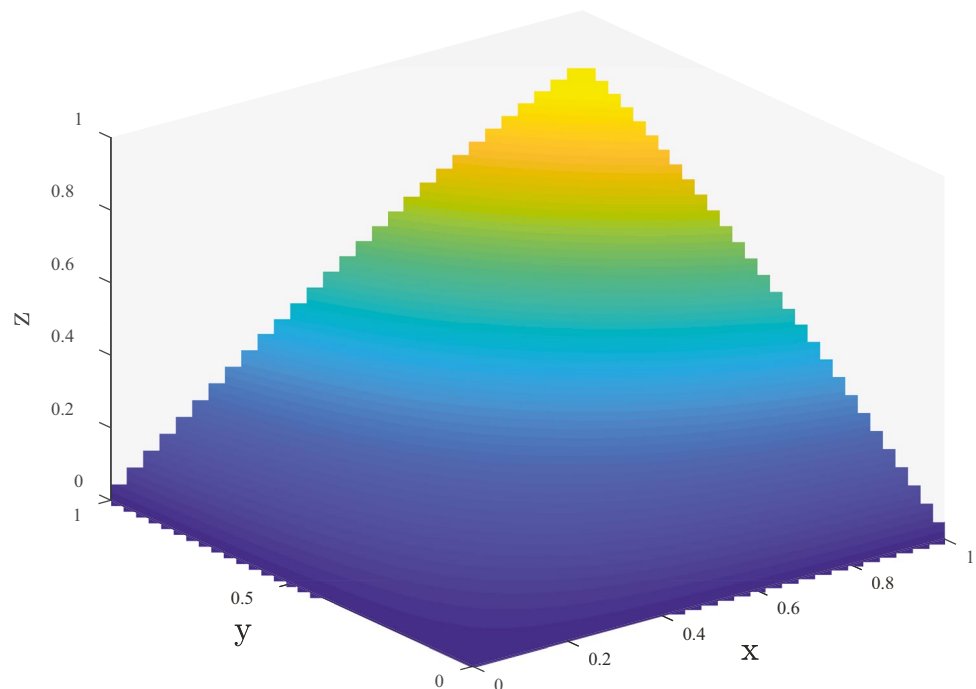
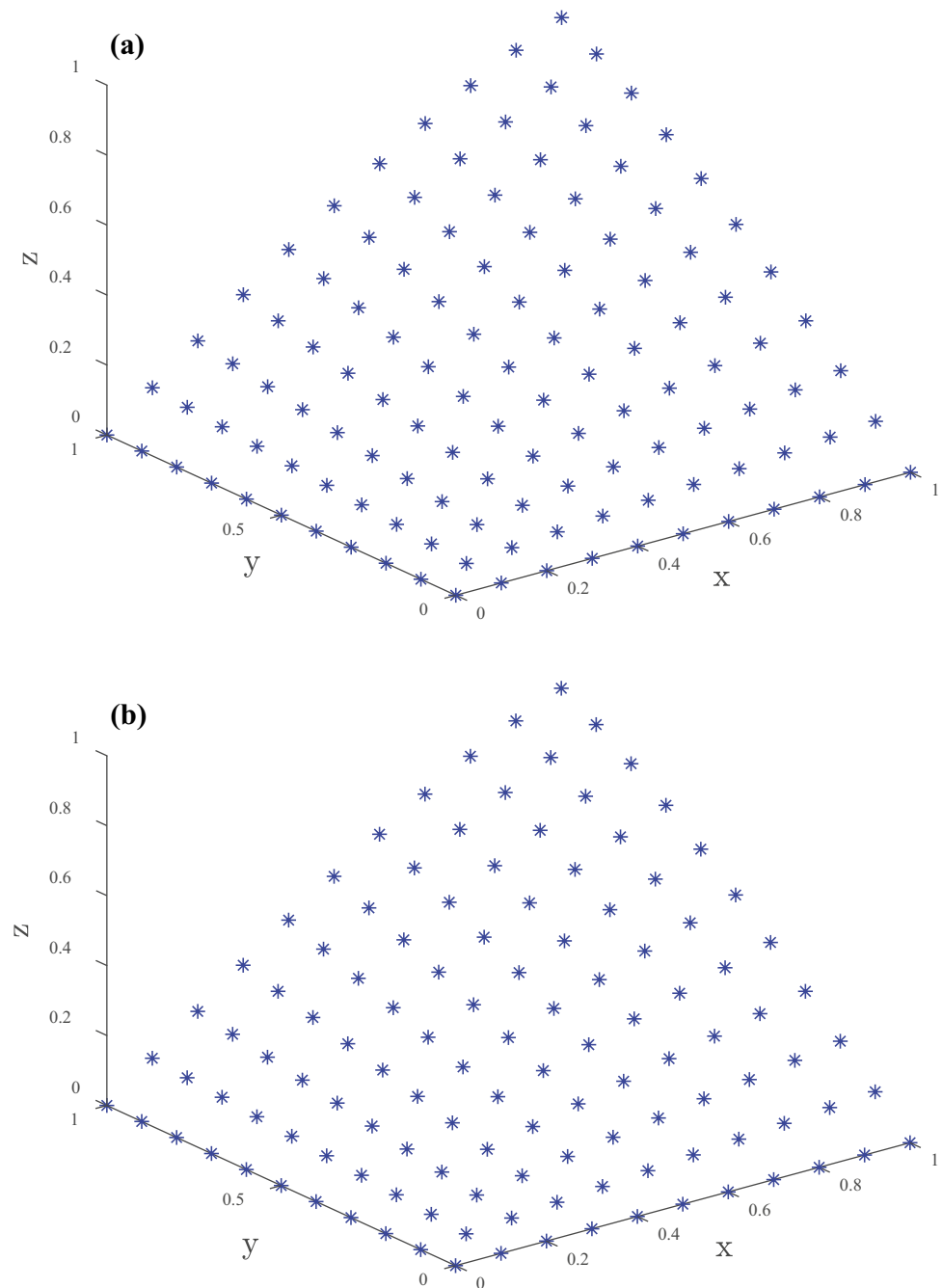


Fig. 7 Solutions to training points in Example 2: **a** exact solution and **b** numerical solution of the proposed model



Example 1, these sampling points are the same as those reported in the literature to fairly evaluate the performance of each scheme. Table 2 shows that the AE values of the KELM vary between 10^{-7} and 10^{-6} , the AE values of the WNNPSO vary between 10^{-6} and 10^{-5} , the AE values of the WNNBOA vary between 10^{-5} and 10^{-3} , and the AE values of the WNNIBOA vary between 10^{-7} and 10^{-5} . According to Table 2, the MAEs of the WNNPSO, WNNBOA,

WNNIBOA, and KELM are $3.07\text{E-}05$, $5.29\text{E-}04$, $6.17\text{E-}06$, and $3.80\text{E-}06$, respectively. The MSEs of the WNNPSO, WNNBOA, WNNIBOA, and KELM are $1.35\text{E-}09$, $4.74\text{E-}07$, $5.66\text{E-}11$, and $2.08\text{E-}11$, respectively. The SDs of the WNNPSO, WNNBOA, WNNIBOA, and KELM are $2.01\text{E-}05$, $4.41\text{E-}04$, $4.30\text{E-}06$, and $2.52\text{E-}06$, respectively. The performances of the five methods from high to low are in the order the KELM, WNNIBOA, WNNPSO, and WNNBOA.

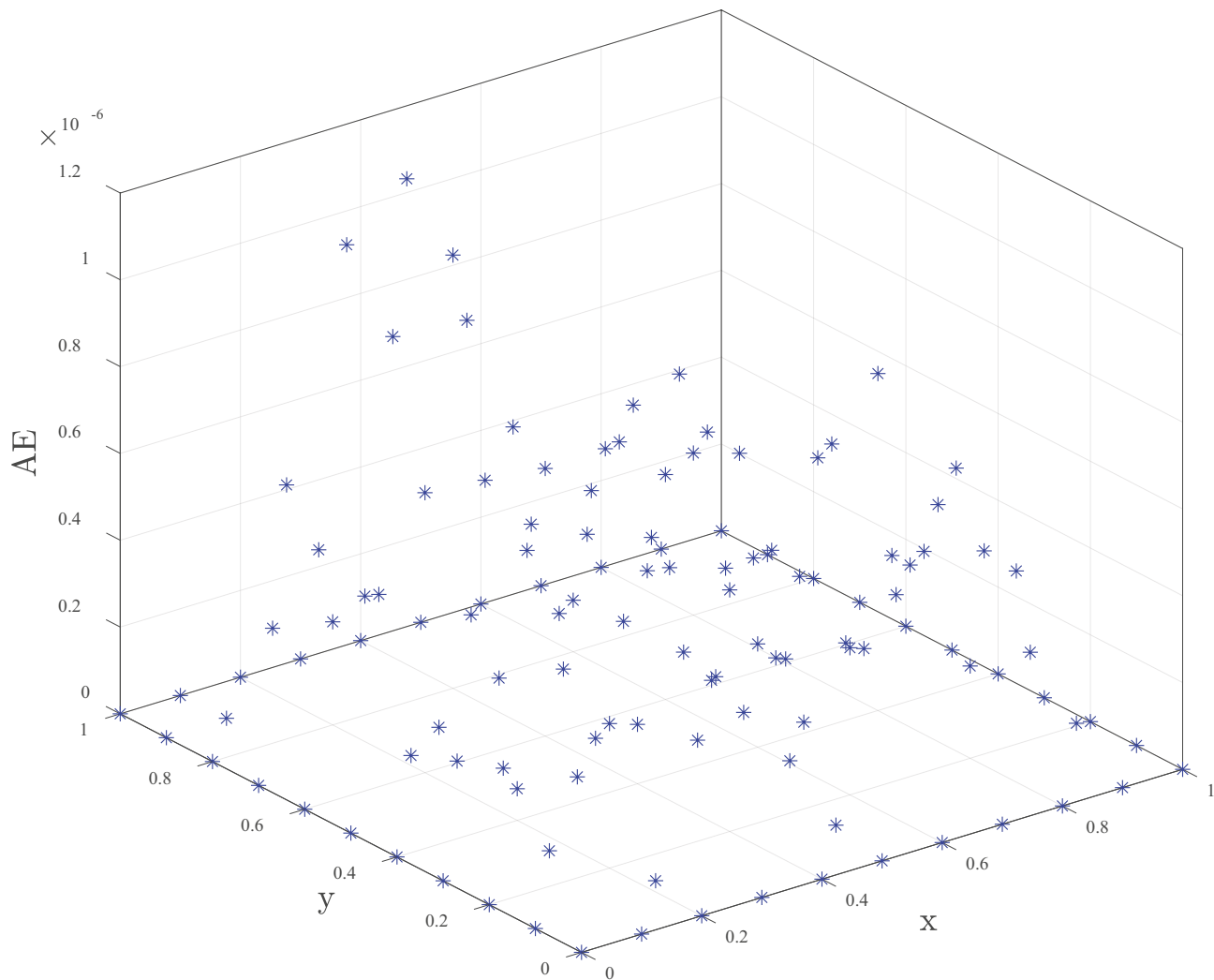


Fig. 8 Absolute error values of training points in Example 2

Example 3 (Mall and Chakraverty [29])

We considered the following PDE:

$$\frac{\partial^2 z}{\partial y^2} + \frac{\partial^2 z}{\partial x^2} = \sin(\pi y)\sin(\pi x), \quad 0 \leq x \leq 1, \quad 0 \leq y \leq 1 \quad (31)$$

with Dirichlet boundary conditions

$$z(0, y) = 0, z(1, y) = 0, z(x, 0) = 0, z(x, 1) = 0$$

According to Eq. (13), the trial solution to Eq. (31) is expressed as

$$z = x(1 - x)y(1 - y)u(x, y)$$

The exact solution is $\hat{z} = -12\pi^2 \sin(\pi x)\sin(\pi y)$ (Fig. 9).

In this example, the ChNN was compared with the KELM.

The sampling points are the same as those in Example 1. Table 3 shows the performance comparison of the KELM and ChNN on untrained points (the calculation results of the ChNN are from Mall and Chakraverty [29]). Table 3

Table 2 Absolute errors of the four methods

x	y	WNNPSO [8]	WNNBOA [8]	WNNIBOA [8]	KELM
0	0.891	0	0	0	0
0.171	0.077	2.46E-05	3.11E-04	7.99E-06	2.39E-06
0.39	0.194	4.08E-05	1.14E-03	1.07E-05	5.95E-06
0.47	0.284	3.41E-05	1.46E-03	1.48E-05	7.03E-06
0.825	0.539	3.26E-05	7.17E-04	3.85E-06	1.83E-06
0.34	0.682	3.26E-05	3.49E-04	7.65E-06	7.49E-07
0.741	0.568	5.03E-05	5.17E-04	2.61E-06	7.95E-06
0.95	0.394	1.27E-05	3.18E-04	6.99E-06	3.07E-06
0.153	0.882	7.20E-05	2.08E-05	9.09E-07	4.31E-06
0.937	0.472	7.75E-06	4.53E-04	6.19E-06	4.71E-06

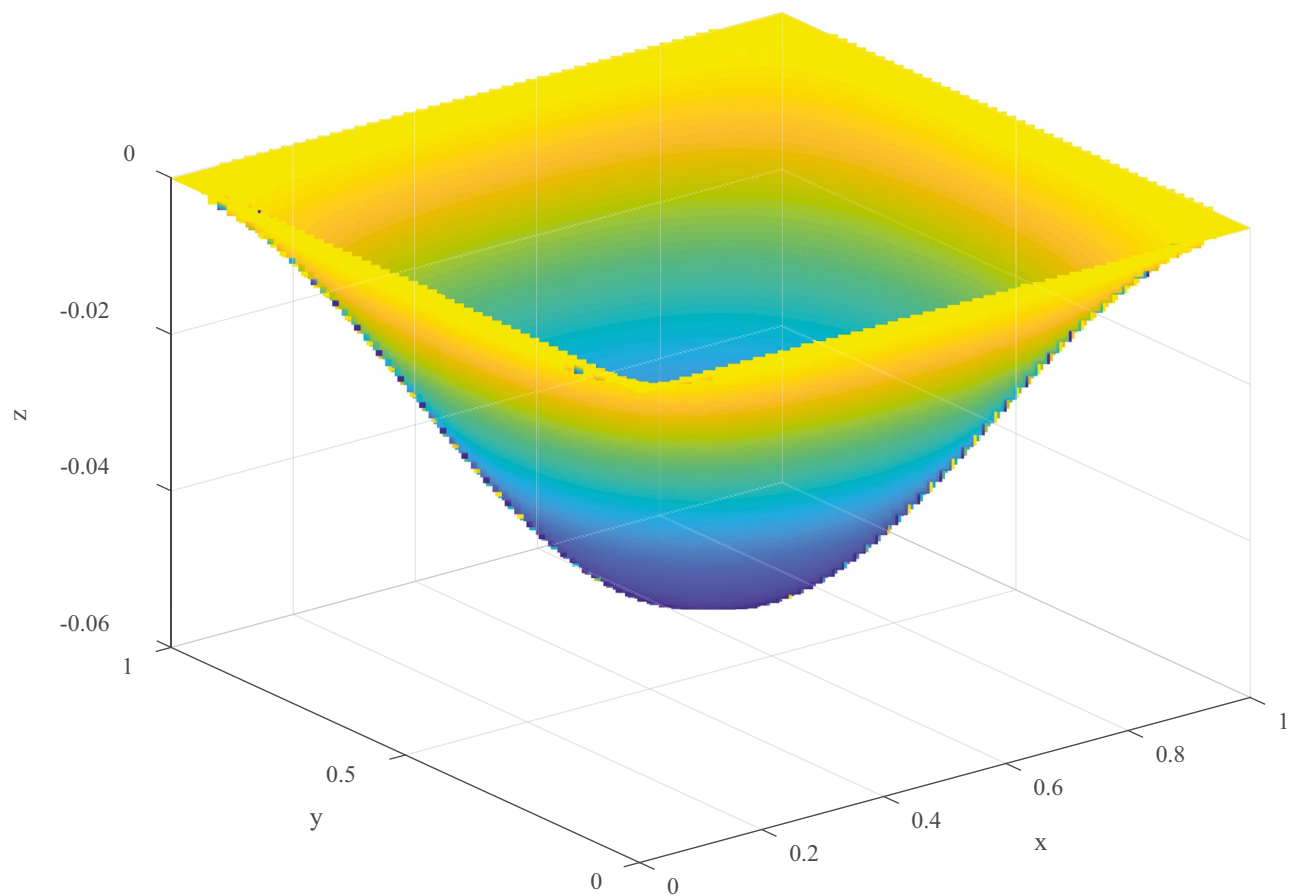


Fig. 9 Exact solution to Eq. (31)

shows that the AE values of the KELM vary between 10^{-7} and 10^{-6} , and the AE values of the ChNN vary between 10^{-4} and 10^{-3} . According to Table 3, the MAEs of the ChNN and KELM are 2.00E-03 and 3.39E-06, respectively. The MSEs of the ChNN and KELM are 6.32E-06 and 1.73E-11, respectively. The SDs of the ChNN and KELM are 1.52E-03 and 2.42E-06, respectively. The performance of the KELM is still better than that of ChNN.

Table 3 Absolute errors of the two methods

<i>x</i>	<i>y</i>	ChNN [29]	KELM
0	0.891	0	0
0.171	0.077	2.54E-03	1.70E-06
0.39	0.194	4.75E-03	5.68E-06
0.47	0.284	4.68E-04	6.89E-06
0.825	0.539	2.97E-03	1.65E-06
0.34	0.682	2.27E-04	6.95E-07
0.741	0.568	1.46E-03	7.34E-06
0.95	0.394	2.69E-03	2.51E-06
0.153	0.882	1.08E-03	3.86E-06
0.937	0.472	3.81E-03	3.54E-06

Discussion

As seen from the above three cases, the KELM had better performance than the state-of-the-art methods. In this section, we will expand the discussion on the effect of grid size (i.e., the number of training points) on the KELM. A 5×5 grid (with

Table 4 Effect of the training point number on the performance of the proposed method

Example	Index	Number of training points			
		36	121	256	441
1	MAE	1.32E-04	1.04E-06	9.58E-07	9.39E-07
	MSE	2.30E-08	2.92E-12	1.43E-12	1.38E-12
	SD	2.65E-04	1.35E-06	1.02E-06	9.85E-07
2	MAE	1.65E-04	3.80E-06	3.61E-06	3.52E-06
	MSE	5.21E-08	2.08E-11	2.02E-11	1.95E-11
	SD	9.59E-05	2.52E-06	2.42E-06	2.36E-06
3	MAE	1.47E-04	3.39E-06	3.19E-06	3.12E-06
	MSE	9.41E-08	1.73E-11	1.66E-11	1.63E-11
	SD	1.14E-04	2.42E-06	2.30E-06	2.25E-06

36 training points), 10×10 grid (with 121 training points), 15×15 grid (with 256 training points), and 20×20 grid (with 441 training points) were used to divide the definition domain of the above three cases. The MAE, MSE, and SD values of the KELM obtained are shown in Table 4. As the grid was encrypted, the MAE, MSE, and SD of the proposed method decreased. However, the performance of the proposed method with the use of a 10×10 grid was very close to that of a 20×20 grid. Considering that a grid that is too dense will increase the complexity of the algorithm, grid meshing should consider both accuracy and complexity, and the method in this paper only needs a few training points to achieve high computational accuracy. Thus, the advantage of the highlighted proposed method is that only a few training points are needed to achieve high computational accuracy.

Application

Water flow in unsaturated soils is an important issue in engineering geology [31]. The proposed method focused on solving this problem, considering a simple steady-state scenario.

The geometry and boundary conditions are illustrated in Fig. 10, where $L = 1$ m and $W = 1$ m. Silt was adopted in this case. The governing equation is described as follows [32]:

$$\frac{\partial}{\partial x} \left(K_x(h) \frac{\partial h}{\partial x} \right) + \frac{\partial}{\partial y} \left(K_y(h) \frac{\partial h}{\partial y} + K_y(h) \right) = 0 \quad (32)$$

$$K(h) = K_s e^{\alpha_g h} \quad (33)$$

where K_s is the saturated permeability coefficient of soils ($K_s = 1 \times 10^{-4} \text{ m/s}$); h is the water head; and α_g is the pore-size distribution parameter ($\alpha_g = 8 \times 10^{-3}$). The boundaries of this equation can be described as follows ($h_d = -1 \text{ m}$):

$$h(0, y) = h_d \quad (34)$$

$$h(W, y) = h_d \quad (35)$$

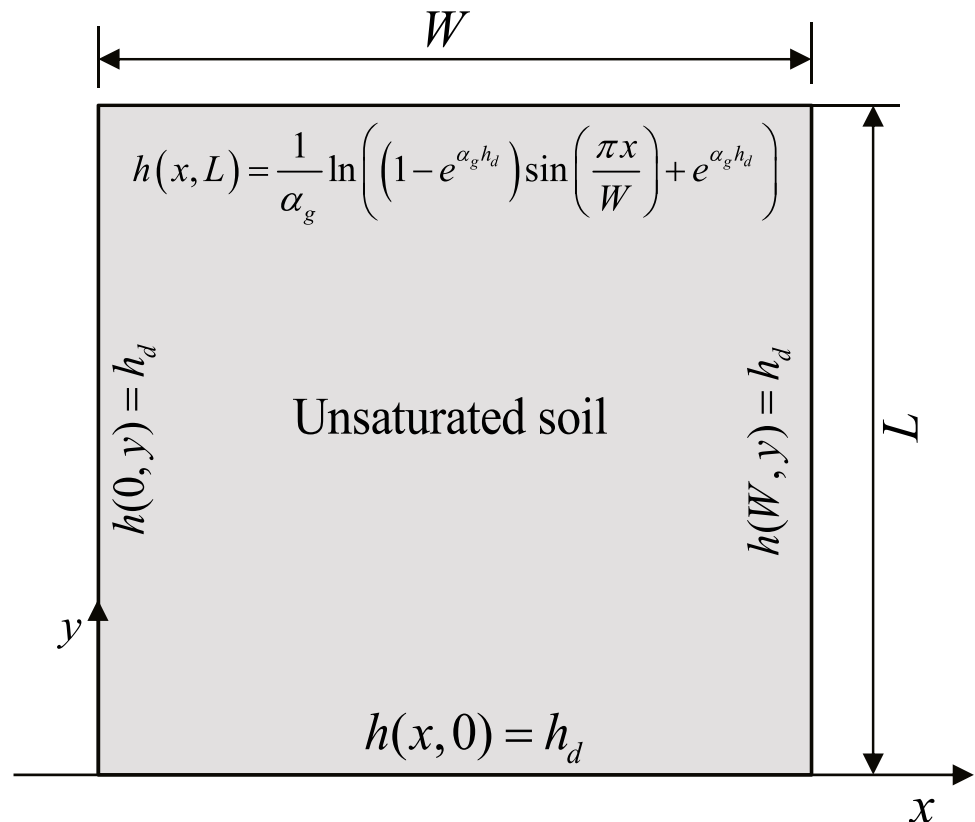
$$h(x, 0) = h_d \quad (36)$$

$$h(x, L) = \ln \left((1 - e^{\alpha_g h_d}) \sin \left(\frac{\pi x}{W} \right) + e^{\alpha_g h_d} \right) / \alpha_g \quad (37)$$

The analytical solution to Eqs. (32) and (33) can be expressed as (Tracy [33])

$$h(x, y) = \frac{1}{\alpha_g} \ln \left[e^{\alpha_g h_d} + (1 - e^{\alpha_g h_d}) e^{\frac{\alpha_g (L-y)}{2}} \sin \left(\frac{\pi x}{W} \right) \frac{\sinh \left(y \sqrt{\frac{\alpha_g^2}{4} + \left(\frac{\pi}{W} \right)^2} \right)}{\sinh \left(L \sqrt{\frac{\alpha_g^2}{4} + \left(\frac{\pi}{W} \right)^2} \right)} \right] \quad (38)$$

Fig. 10 Schematic illustration of steady-state flow



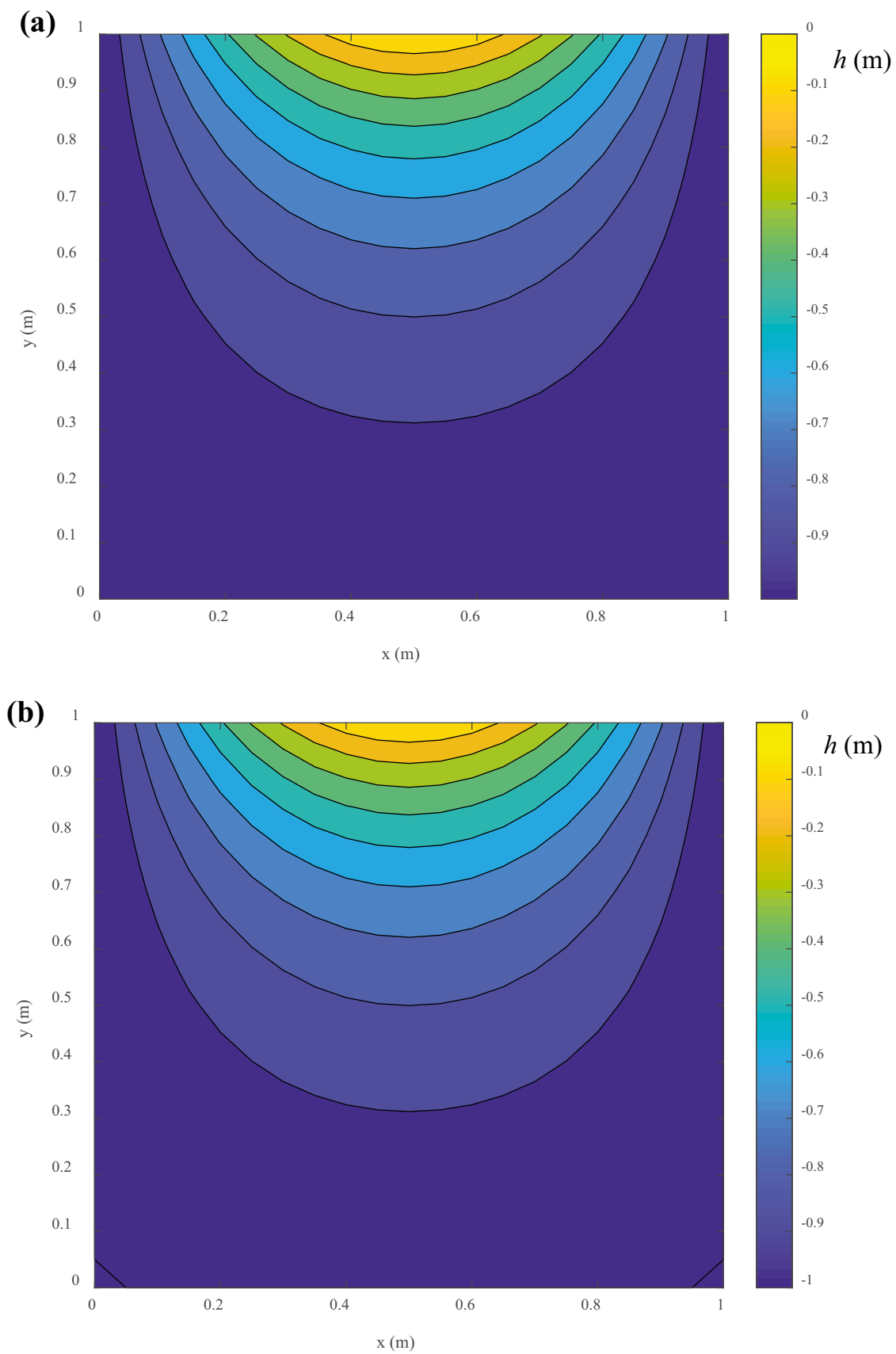


Fig. 11 **a** Analytical solution and **b** numerical solution

Figure 11 reports the analytical solution to Eqs. (32) and (33) and the solution obtained by the proposed method. The calculated results of the numerical solution are very consistent with those of the analytical solution. This method makes the constrained optimization problem a much simpler unconstrained optimization problem and exhibits excellent computational accuracy. The successful application to this case demonstrates the great value of the proposed method.

Conclusions

In this study, inspired by extreme learning machines with kernels, a new method for solving elliptic partial differential equations with Dirichlet boundary conditions is presented. Benefitting from the contribution of the constructed trial solutions and the powerful approximation abilities of extreme learning machines with kernels, the errors of the proposed method in the three cases (mean absolute error, mean squared error, and standard deviation) were all smaller than those of the existing methods, including the wavelet neural network optimized with the improved butterfly optimization algorithm. The proposed KELM method was successfully applied to solve a water flow problem in unsaturated soils. Two significant advantages of the proposed method over the existing methods were highlighted: (1) fewer parameters (only two parameters) and (2) fewer training points are needed.

Appendix

A1 Expressions of T_j in Eq. (21) and Ω_{ij} in Eq. (23)

In Eq. (21),

$$\frac{\partial A(X_j)}{\partial x} = -\frac{h_0(y_j)}{b-a} + \frac{h_1(y_j)}{b-a} + \frac{(d-y_j)}{d-c} \left\{ \frac{\partial q_0(x_j)}{\partial x} - \left[-\frac{q_0(a)}{b-a} + \frac{q_0(b)}{b-a} \right] \right\} \\ + \frac{y_j-c}{d-c} \left\{ \frac{\partial q_1(x_j)}{\partial x} - \left[-\frac{q_1(a)}{b-a} + \frac{q_1(b)}{b-a} \right] \right\}$$

$$\frac{\partial^2 A(X_j)}{\partial x^2} = \frac{(d-y_j)}{d-c} \left\{ \frac{\partial^2 q_0(x_j)}{\partial x^2} \right\} + \frac{y_j-c}{d-c} \left\{ \frac{\partial^2 q_1(x_j)}{\partial x^2} \right\}$$

$$\frac{\partial A(X_j)}{\partial y} = \frac{(b-x_j)}{b-a} \frac{\partial h_0(y_j)}{\partial y} + \frac{x_j-a}{b-a} \frac{\partial h_1(y_j)}{\partial y} \\ - \frac{1}{d-c} \left\{ q_0(x_j) - \left[\frac{(b-x_j)}{b-a} q_0(a) + \frac{x_j-a}{b-a} q_0(b) \right] \right\} \\ + \frac{1}{d-c} \left\{ q_1(x_j) - \left[\frac{(b-x_j)}{b-a} q_1(a) + \frac{x_j-a}{b-a} q_1(b) \right] \right\}$$

$$\frac{\partial^2 A(X_j)}{\partial y^2} = \frac{(b-x_j)}{b-a} \frac{\partial^2 h_0(y_j)}{\partial y^2} + \frac{x_j-a}{b-a} \frac{\partial^2 h_1(y_j)}{\partial y^2}$$

In Eq. (23),

$$K(X_i, X_j) = \exp\left(-\frac{\|X_i - X_j\|^2}{\gamma^2}\right)$$

$$\frac{\partial K(X_i, X_j)}{\partial x} = -2(x_i - x_j) \exp\left(-\frac{\|X_i - X_j\|^2}{\gamma^2}\right) / \gamma^2$$

$$\frac{\partial K(X_i, X_j)}{\partial y} = -2(y_j - y_i) \exp\left(-\frac{\|X_i - X_j\|^2}{\gamma^2}\right) / \gamma^2$$

$$\frac{\partial^2 K(X_i, X_j)}{\partial x^2} = -2(\gamma^2 - 2x_j^2 + 4x_jx_i - 2x_i^2) \exp\left(-\frac{\|X_i - X_j\|^2}{\gamma^2}\right) / \gamma^4$$

$$\frac{\partial^2 K(X_i, X_j)}{\partial y^2} = -2(\gamma^2 - 2y_j^2 + 4y_jy_i - 2y_i^2) \exp\left(-\frac{\|X_i - X_j\|^2}{\gamma^2}\right) / \gamma^4$$

$$\frac{\partial B(X_j)}{\partial x} = -(c - y_j)(d - y_j)(a + b - 2x_j)$$

$$\frac{\partial B(X_j)}{\partial y} = -(a - x_j)(b - x_j)(c + d - 2y_j)$$

$$\frac{\partial^2 B(x_j, y_j)}{\partial x^2} = 2(c - y_j)(d - y_j)$$

$$\frac{\partial^2 B(X_j)}{\partial y^2} = 2(a - x_j)(b - x_j)$$

A2 Brief Introductions to the Methods Used for Comparison

ChNN: This method used the Chebyshev neural network to construct a trial solution and used a gradient-based optimization algorithm to learn the connection weight and bias of the network.

WNNPSO: The wavelet neural network was used to construct the trial solution. The particle swarm optimization algorithm was used as the learning algorithm to obtain the connection weights and biases of the wavelet neural network. The particle swarm optimization algorithm is a classic heuristic optimization algorithm pioneered by Kennedy and Eberhart [34] and has been widely used in practice [35].

WNNBOA: The wavelet neural network was used to construct the trial solution. The butterfly optimization algorithm [36, 37] was used to learn the connection weights and biases of wavelet neural networks.

WNNIBOA: The wavelet neural network was used to construct the trial solution. The improved butterfly optimization algorithm was used to learn the connection weights and biases of wavelet neural networks. Compared with the standard butterfly optimization algorithm, the adjustment parameters in the improved butterfly optimization algorithm change dynamically, which theoretically enhances the algorithm's global optimization capability.

The above four methods all need to determine a large number of parameters (network connection weight and bias).

Acknowledgements This work was supported by the National Natural Science Foundation of China (Grant No. 51578466).

Data Availability All data generated or analyzed during this study are included within the article.

Declarations

Conflict of Interest The authors declare no conflict of interest.

References

- Alzubi JA, Alzubi OA, Chen TM. Forward error correction based on algebraic-geometric theory. Springer. 2014.
- Collins BD, Znidarcic D. Stability analyses of rainfall induced landslides. *Journal of Geotechnical and Geoenvironmental Engineering*. 2004;130(4):362–72.
- Iverson RM. Landslide triggering by rain infiltration. *Water Resour Res*. 2000;36(7):1897–910.
- Lu N, Godt J W. Hillslope hydrology and stability. Cambridge University Press. 2013.
- Schmidhuber J. Deep learning in neural networks: an overview. *Neural Netw*. 2015;61:85–117.
- Smola AJ, Schölkopf B. A tutorial on support vector regression. *Stat Comput*. 2004;14(3):199–222.
- Vapnik VN. An overview of statistical learning theory. *IEEE Trans Neural Networks*. 1999;10(5):988–99.
- Tan LS, Zainuddin Z, Ong P. Wavelet neural networks based solutions for elliptic partial differential equations with improved butterfly optimization algorithm training. *Appl Soft Comput*. 2020;95:106518.
- Lagaris IE, Likas A, Fotiadis DI. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Trans Neural Networks*. 1998;9(5):987–1000.
- Lagaris IE, Likas AC, Papageorgiou DG. Neural-network methods for boundary value problems with irregular boundaries. *IEEE Trans Neural Networks*. 2000;11(5):1041–9.
- Alli H, Uçar A, Demir Y. The solutions of vibration control problems using artificial neural networks. *J Franklin Inst*. 2003;340(5):307–25.
- Filici C. On a neural approximator to ODEs. *IEEE Trans Neural Networks*. 2008;19(3):539–43.
- Effati S, Pakdaman M. Artificial neural network approach for solving fuzzy differential equations. *Inf Sci*. 2010;180(8):1434–57.
- Alzubi OA, Alzubi JA, Dorgham O, Alsayyed M. Cryptosystem design based on Hermitian curves for IoT security. *J Supercomput*. 2020;76:8566–89.
- Alzubi OA, Alzubi JA, Tedmori S, Rashaideh H, Almomani O. Consensus-based combining method for classifier ensembles. *The International Arab Journal of Information Technology*. 2018;15(1):76–86.
- Gheisari M, Panwar D, Tomar P, Harsh H, Zhang X, Solanki A, Alzubi JA. An optimization model for software quality prediction with case study analysis using MATLAB. *IEEE Access*. 2019;7:85123–38.
- Huang GB, Zhu QY, Siew CB. Extreme learning machine: a new learning scheme of feedforward neural networks. *IEEE International Joint Conference on Neural Networks (IEEE Cat. No.04CH37541)* 2004;2:985–90.
- Huang GB, Zhu QY, Siew CB. Extreme learning machine: Theory and applications. *Neurocomputing*. 2006;70:489–501.
- Huang FM, Huang JS, Jiang SH, Zhou CB. Landslide displacement prediction based on multivariate chaotic model and extreme learning machine. *Eng Geol*. 2017a;218:173–86.
- Huang FM, Yin KL, Huang JS, Gui L, Wang P. Landslide susceptibility mapping based on self-organizing-map network and extreme learning machine. *Eng Geol*. 2017b;223:11–22.
- Liu C, Sun B, Zhang CH, Li F. A hybrid prediction model for residential electricity consumption using holt-winters and extreme learning machine. *Appl Energy*. 2020;275:115383.
- Albu F, Hagiescu D, Vladutu L, Puica MA. Neural network approaches for children's emotion recognition in intelligent learning applications. In *EDULEARN15 7th Annu Int Conf Educ New Learn Technol Barcelona, Spain, 6th-8th*. 2015.
- Malathi V, Marimuthu NS, Baskar S, Ramar K. Application of extreme learning machine for series compensated transmission line protection. *Eng Appl Artif Intell*. 2011;24(5):880–7.
- Huang GB. An Insight into Extreme Learning Machines: Random Neurons. *Random Features and Kernels Cognitive Computation*. 2014;6(3):376–90.
- Zhou C, Yin KL, Cao Y, Intrieri E, Ahmed B, Catani F. Displacement prediction of step-like landslide by applying a novel kernel extreme learning machine method. *Landslides*. 2018;15(11):2211–25.
- Alizamir M, Kim S, Zounemat-Kermani M, Heddami S, Kim NW, Singh VP. Kernel extreme learning machine: an efficient model for estimating daily dew point temperature using weather data. *Water*. 2020;12(9):2600.
- Li YF, Shi HP, Liu H. A hybrid model for river water level forecasting: cases of Xiangjiang River and Yuanjiang River, China. *J Hydrol*. 2020;124934.
- Chen SY, Gu CS, Lin CN, Hariri-Ardebili MA. Prediction of arch dam deformation via correlated multi-target stacking. *Appl Math Model*. 2021;91:1175–93.
- Mall S, Chakraverty S. Single layer Chebyshev neural network model for solving elliptic partial differential equations. *Neural Process Lett*. 2017;45(3):825–40.
- Peng X. TSVR: an efficient twin support vector machine for regression. *Neural Netw*. 2010;23(3):365–72.
- Javadi AA, Ahangar-Asr A, Johari A, Faramarzi A, Toll D. Modelling stress-strain and volume change behaviour of unsaturated soils using an evolutionary based data mining technique, an incremental approach. *Eng Appl Artif Intell*. 2012;25(5):926–33.
- Wu LZ, Zhu SR, Peng PJ. Application of the Chebyshev spectral method to the simulation of groundwater flow and rainfall-induced landslides. *Appl Math Model*. 2020;80:408–25.
- Tracy FT. Clean two- and three-dimensional analytical solutions of Richards' equation for testing numerical solvers. *Water Resour Res*. 2006;42(8):W08503.

34. Kennedy J, Eberhart R. Particle swarm optimization. Proceedings of IEEE International Conference Neural Networks, IEEE Service Center. 1995;1995:1942–8.
35. Chau KW. Particle swarm optimization training algorithm for ANNs in stage prediction of Shing Mun River. J Hydrol. 2006;329(3–4):363–7.
36. Arora S, Singh S. Butterfly optimization algorithm: a novel approach for global optimization. Soft Comput. 2019;23:715–34.
37. Wen L, Cao Y. A hybrid intelligent predicting model for exploring household CO₂ emissions mitigation strategies derived from butterfly optimization algorithm. Sci Total Environ. 2020;727:138572.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.